

Open in app ↗

Sign up

Sign in

Medium

 Search

Routing Your Home Lab to AWS with Tailscale: VPN via CGNAT and no Static IP needed

15 min read · May 5, 2025



Alberto Crescini

Follow



Listen



Share



So you want to build your own Hybrid EKS Cluster as we spoke on [this previous article](#)? Or maybe you're looking to access an Amazon RDS instance sitting quietly in a private subnet without punching holes in your VPC or

routing all your traffic through the open internet?

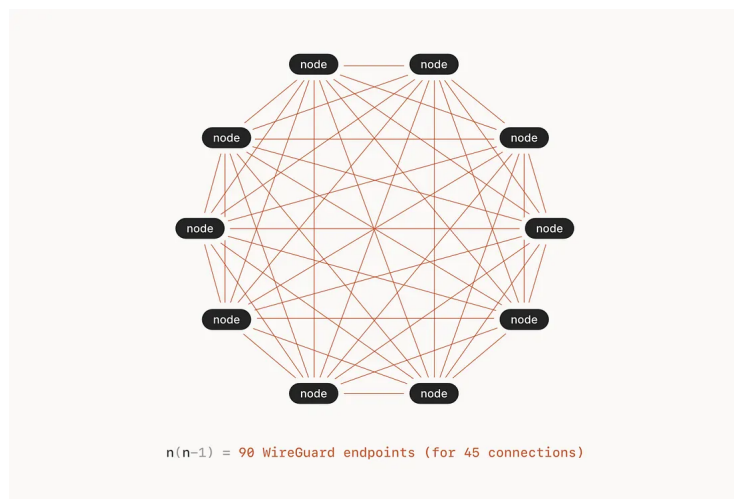
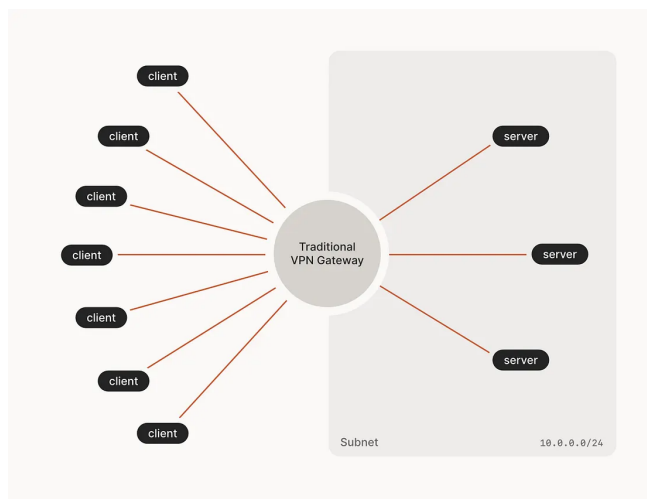
There are plenty of options available. At the top end, you have **AWS Direct Connect** definitely not for our home setup. A more balanced alternative is **Site-to-Site VPN**, which requires a **static IP** and some upfront configuration effort. You could even roll your own VPN server on an EC2 instance, choosing your preferred VPN protocol and defining custom routes into your VPC potentially saving on costs if you size your EC2 right.

But what if your ISP doesn't give you a public IP? What if you're buried behind multiple layers of NAT?

In this post, I'll show you how I built a secure bridge between my home network and AWS using **Tailscale**, enabling a hybrid Kubernetes setup where **EKS provides the control plane** and a **Raspberry Pi 5 runs as a worker node**. All of it works **without a public IP** and **without costly infrastructure**.

What is Tailscale

Tailscale is a modern VPN built on top of **WireGuard**, designed to create secure, peer-to-peer networks between your devices — no matter where they are or how many NAT layers they're behind. Unlike traditional VPNs that route all traffic through centralized servers, Tailscale builds a **mesh network**, allowing devices to communicate directly when possible, or relay efficiently when not. It's a **managed VPN service** with a built-in control plane for device authentication and key exchange. You can log in using identity providers like GitHub or Google, and Tailscale takes care of coordinating encrypted metadata — **without ever routing your actual traffic through its infrastructure**. Your traffic remains **end-to-end encrypted**, device to device.

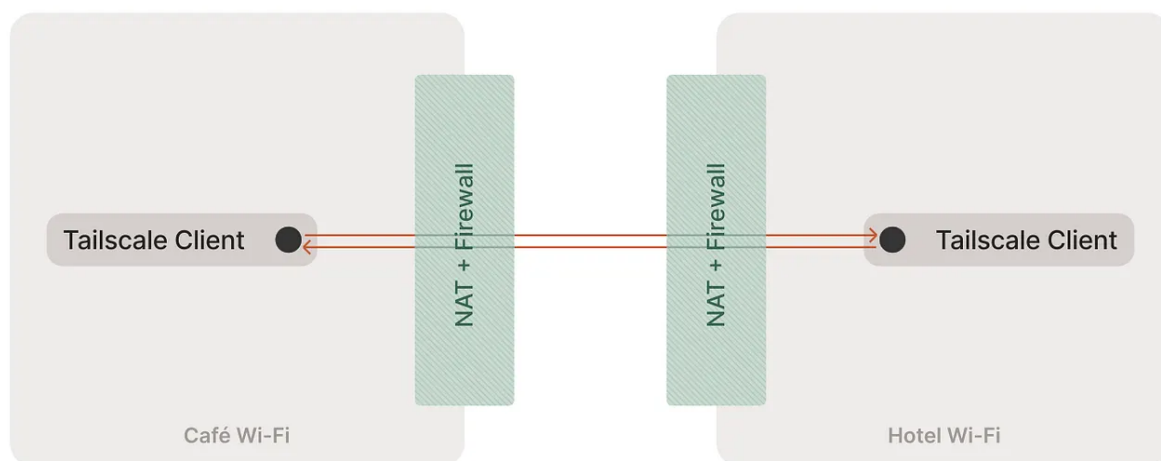


Traditional VPN server vs Tailscale Mesh network

What makes Tailscale stand out is its ability to **navigate difficult network topologies**, like double NAT, carrier-grade NAT (CGNAT), and firewalls that block unsolicited inbound traffic. To do this, Tailscale leverages the same techniques used in WebRTC: namely **STUN** and **ICE**.

In simple terms, Tailscale attempts **NAT traversal** by having both peers simultaneously initiate outbound connections, punching symmetric holes in their respective NAT devices. If this works, and it often does, the devices connect directly, even without any port forwarding.

But not all networks play nice. Some environments block **UDP entirely**, use **symmetric NATs**, or introduce too many layers of complexity for direct connection. In those scenarios **DERP** (Designated Encrypted Relay for Packets) comes in, acting as encrypted TCP relays between devices that can't reach each other directly. Unlike traditional TURN servers, DERP uses **HTTPS streams** and **WireGuard keys** for secure, modern relaying. Even then, all traffic remains **end-to-end encrypted**, DERP simply forwards encrypted packets, without visibility into their content.



Tailscale architecture through one NAT Layer

Once we've set up **two Tailscale nodes**, one at home (e.g., a Raspberry Pi) and one inside the **AWS VPC** (e.g., an EC2 instance), we'll configure **routing rules** so traffic from either side can be forwarded through these nodes.

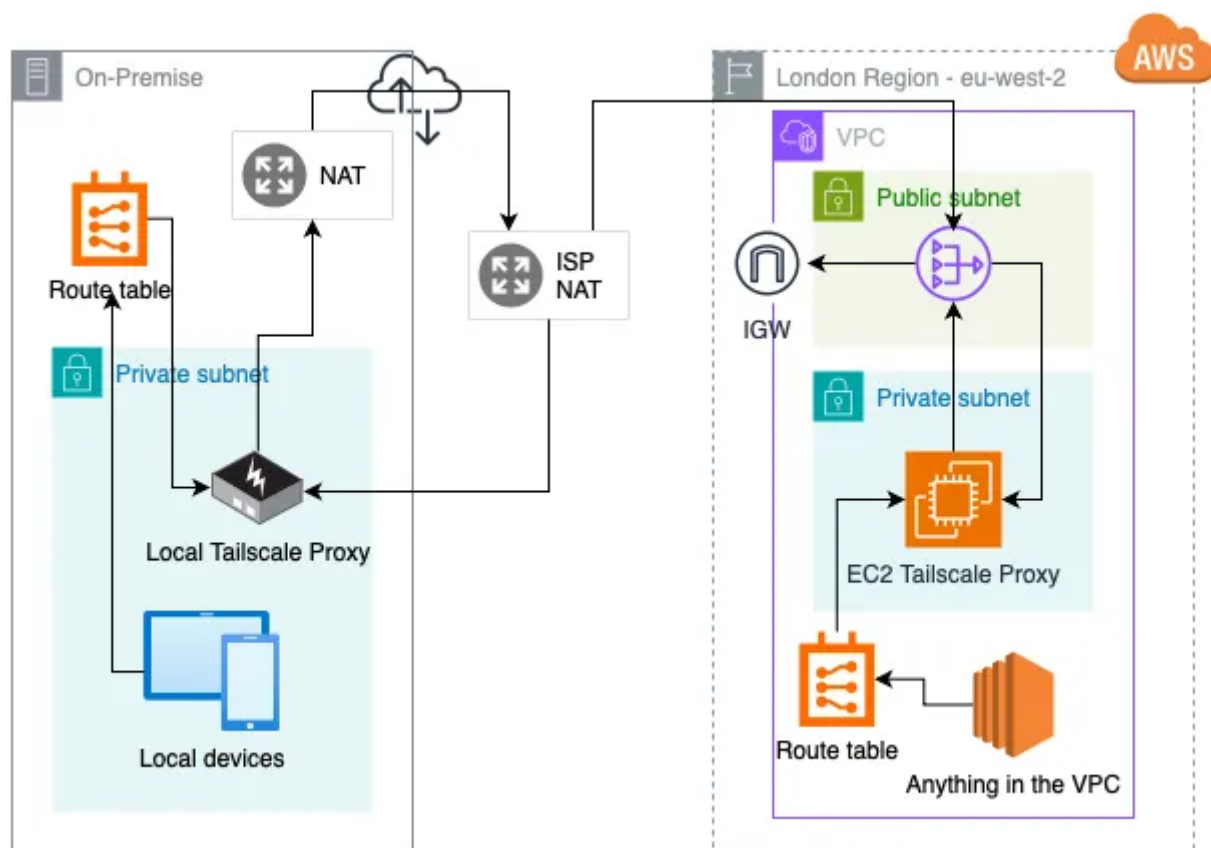
With this setup in place, you'll be able to:

- Access internal AWS resources (like an **RDS database** in a private subnet) from any device on your home network, without needing a VPN client or public IP.
- Allow services in your AWS VPC to securely connect to devices on your local network (e.g., a **NAS file share**), even if you're behind multiple layer of NAT from your ISP.

In short, Tailscale acts as a secure bidirectional bridge, eliminating the complexity of traditional VPNs and making hybrid networking approachable.

Overview

Our goal is to create two Tailscale-connected nodes: one running at home (for example, on a Raspberry Pi), and another within the AWS VPC (typically an EC2 instance). These two nodes will act as **relays** between your local network and AWS. Once connected, we'll configure **routing rules** to allow traffic from your home network to reach the VPC via the EC2 node, the final infrastructure will look something like this:



Starting from the AWS side, we'll deploy a low-cost `t4g.nano` EC2 instance in a **private subnet** within the VPC. This instance will not expose any ports to the public internet for security reasons. However, in order to install required packages and join the Tailscale network, it still needs **outbound internet access**.

While you *could* assign a public IP directly to the instance and route traffic through an Internet Gateway, doing so would violate the isolation of the private subnet, essentially converting it into a public subnet. Instead, we'll preserve the private nature of the subnet and use a **NAT Gateway** deployed in a public subnet. This allows the EC2 instance to reach the internet for outbound connections (e.g., Tailscale control plane, software installs), while still remaining inaccessible from the outside world. This approach also allows for centralized egress control, consistent IP address usage, and better security posture.

On the on-premises side, we'll use a **Raspberry Pi** (or any low-power Linux device) running the Tailscale daemon. This node will act as the gateway for your home network, and we'll configure the local routing table so that traffic destined for AWS subnets (e.g., the VPC CIDR or private service IPs) is forwarded through

this Pi, over the encrypted Tailscale tunnel

Configure the AWS resources

Deploy the VPC

Let's start by creating a VPC. You can use the following terraform script to recreate a VPC that spans across three Availability Zones, with two public subnets and three private ones. It also creates a NAT Gateway instance in the first public subnet.

```
## variables.tf ##

# Configure your AWS infrastructure here
# Adjust these variables to deploy in your preferred region

variable "region" {
  description = "AWS region where resources will be deployed"
  type        = string
  default     = "eu-west-2" # Change to your preferred region
}

variable "availability_zones" {
  description = "Availability Zone suffixes for multi-AZ deployment"
  type        = list(string)
  default     = ["a", "b", "c"] # AZ suffixes for the specified region
}

variable "vpc_cidr" {
  description = "Primary CIDR block for the VPC"
  type        = string
  default     = "10.0.0.0/16" # Overall VPC address space
}

variable "public_subnet_count" {
  description = "Number of public subnets to create"
  type        = number
  default     = 2 # Creates public subnets in first 2 AZs
}
```

Feel free to adjust the variable file to deploy these within your favorite region, and to define the VPC CIDR range. You will then create the *main.tf* module that creates the foundational networking components.

```
## main.tf ##

# This creates the foundational networking components:
# - VPC spanning multiple Availability Zones
# - Public and private subnets
# - NAT Gateway for private subnet internet access

# Provider configuration

provider "aws" {
  region = var.region
}

# Core VPC
resource "aws_vpc" "main" {
  cidr_block      = var.vpc_cidr
  enable_dns_support = true
  enable_dns_hostnames = true
  tags = {
    Name = "hybrid-vpc"
  }
}

# Internet Gateway for public subnets
resource "aws_internet_gateway" "igw" {
  vpc_id = aws_vpc.main.id
  tags = {
    Name = "hybrid-igw"
  }
}

# Public Subnets (Internet-facing)
resource "aws_subnet" "public" {
  count = var.public_subnet_count

  vpc_id      = aws_vpc.main.id
  cidr_block  = cidrsubnet(var.vpc_cidr, 8, 254 + count.index)
  availability_zone = "${var.region}${var.availability_zones[count.index]}"
  map_public_ip_on_launch = false

  tags = {
    Name = "hybrid-subnet-public1-${var.region}${var.availability_zones[count.index]}"
  }
}

# Private Subnets
resource "aws_subnet" "private" {
  count = length(var.availability_zones)
```

```

vpc_id          = aws_vpc.main.id
cidr_block      = cidrsubnet(var.vpc_cidr, 4, 8 + count.index)
availability_zone = "${var.region}${var.availability_zones[count.index]}"
tags = {
  Name = "hybrid-subnet-private1-${var.region}${var.availability_zones[count.index]}"
}
}

# NAT Gateway Components
resource "aws_eip" "nat" {
  domain = "vpc"
  tags = {
    Name = "hybrid-nat-eip"
  }
}

resource "aws_nat_gateway" "nat" {
  allocation_id = aws_eip.nat.id
  subnet_id     = aws_subnet.public[0].id # Placed in first public subnet
  tags = {
    Name = "hybrid-nat-gw"
  }
  depends_on = [aws_internet_gateway.igw]
}

```

After creating the subnets, we'll define two separate route tables, one for the public subnets and one for the private ones. The public route table will direct outbound traffic destined for external networks to the Internet Gateway, while the private route table will route such traffic through the NAT Gateway configured earlier.

```

## routing.tf ##

# This configures all routing relationships between components:
# - Public subnet routing through IGW
# - Private subnet routing through NAT Gateway

# Public Route Table
resource "aws_route_table" "public" {
  vpc_id = aws_vpc.main.id
  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.igw.id
  }
  tags = {

```

```
    Name = "hybrid-rt-public"
  }
}

# Associate Public Subnets with Route Table
resource "aws_route_table_association" "public" {
  count          = length(aws_subnet.public)
  subnet_id      = aws_subnet.public[count.index].id
  route_table_id = aws_route_table.public.id
}

# Private Route Table with NAT Gateway route
resource "aws_route_table" "private" {
  vpc_id = aws_vpc.main.id
  route {
    cidr_block      = "0.0.0.0/0"
    nat_gateway_id = aws_nat_gateway.nat.id
  }
  tags = {
    Name = "hybrid-rt-private"
  }
}

# Associate Private Subnets with Route Table
resource "aws_route_table_association" "private" {
  count          = length(aws_subnet.private)
  subnet_id      = aws_subnet.private[count.index].id
  route_table_id = aws_route_table.private.id
}

# Outputs for integration with other components
output "vpc_id" {
  description = "ID of the created VPC"
  value       = aws_vpc.main.id
}

output "private_subnet_ids" {
  description = "IDs of all private subnets"
  value       = aws_subnet.private[*].id
}

output "public_subnet_ids" {
  description = "IDs of all public subnets"
  value       = aws_subnet.public[*].id
}

output "nat_gateway_ip" {
  description = "Public IP of the NAT Gateway"
  value       = aws_eip.nat.public_ip
}
```

Once created, you should see a similar subnet in your AWS account console:

The screenshot displays the AWS Management Console for a VPC named 'vpc-0e0a / hybrid-vpc'. The 'Details' tab is active, showing various configuration options. The 'State' is 'Available'. The 'Main route table' is 'rtb-09c3'. The 'IPv4 CIDR' is '10.0.0.0/16'. The 'Route 53 Resolver DNS Firewall rule groups' are listed as '-'. The 'Resource map' tab is also visible, showing a diagram of the VPC resources. The diagram includes a 'VPC' box labeled 'hybrid-vpc', a 'Subnets (5)' box containing five subnets (two public and three private), a 'Route tables (3)' box containing three route tables (one public and two private), and a 'Network connections (2)' box containing two connections (one to an Internet Gateway and one to a VPC peering connection).

Deploy the proxy instance

Next, we'll launch an EC2 instance to serve as a Tailscale proxy. Since Tailscale has minimal resource requirements and only needs to handle traffic forwarding between AWS resources, a cost-effective ARM-based `t4g.nano` instance is sufficient running at under \$3.40/month. Select Ubuntu Server as the operating system, and deploy the instance into one of the private subnets—preferably in the same Availability Zone as the NAT Gateway (AZ a) to avoid cross-AZ data transfer charges.

Once provisioned, connect to the instance via SSH using AWS Systems Manager Session Manager. You can then install Tailscale with the following command:

```
curl -fsSL https://tailscale.com/install.sh | sh
```

We now need to configure Tailscale to designate this instance as the subnet router for our VPC CIDR range. This enables all other nodes in the Tailscale network to route traffic to AWS resources within the VPC via this instance.

Ideally, the instance should accept and advertise all current and future routes on the Tailscale network to ensure seamless connectivity across environments.

To do so, just run Tailscale with the following command, replacing the VPC range accordingly:

```
sudo systemctl enable --now tailscaled  
sudo tailscale up --advertise-routes=10.0.0.0/16 --accept-routes
```

Authenticate by logging in with your account, and you should be able to see your device as part of your Tailscale network. Then, *you need then to approve the route from the Tailscale Web UI* to allow the device to start advertising your VPC range.

```
[ssm-user@ip-10-0-130-9 bin]$ tailscale status  
100.125.158.32  aws-exit-node  albertox91@  linux  -
```

The EC2 instance is now part of our Tailscale Network

Now that Tailscale is configured, we need to set up the necessary networking rules to allow the instance to forward traffic. Specifically, we must enable IPv4 forwarding on the instance so it can route packets between network interfaces. This is needed for the instance to act as a proper proxy between Tailscale and the VPC.

To do this, we need to create a new forwarding rule to the system configuration and reload it.

```
echo "net.ipv4.ip_forward = 1" | sudo tee /etc/sysctl.d/99-ip-forward.conf  
sudo sysctl --system
```

To allow your EC2 instance to forward traffic between the Tailscale network and your VPC, you need to configure `iptables` to perform Network Address Translation (NAT) and allow packet forwarding:

```
iptables -t nat -A POSTROUTING -o tailscale0 -j MASQUERADE
```

This command tells the kernel to rewrite the source IP of packets leaving via the Tailscale interface (`tailscale0`) to the EC2 instance's IP. This is essential for return traffic to know where to go it "masquerades" the source IP with the NAT Gateway one.

```
iptables -A FORWARD -i eth0 -o tailscale0 -j ACCEPT
```

This allows packets coming in on the VPC interface (`eth0`) to be forwarded out to Tailscale. We'll use this rule when other instances will be forwarding traffic towards our instance, so that it could be sent over Tailscale.

```
iptables -A FORWARD -i tailscale0 -o eth0 -m state --state RELATED,ESTABLISHED
```

And always, this allows return traffic from Tailscale back into the VPC, but only for connections that were already initiated (i.e., part of an established session). It helps secure the forwarding rules.

Get Alberto Crescini's stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

☐ Remember me for faster sign in

Now, to make sure these rules persist across reboots we install install a helper to

save and restore iptables rules automatically at boot, saves your current rules to /etc/iptables/rules.v4 and /etc/iptables/rules.v6

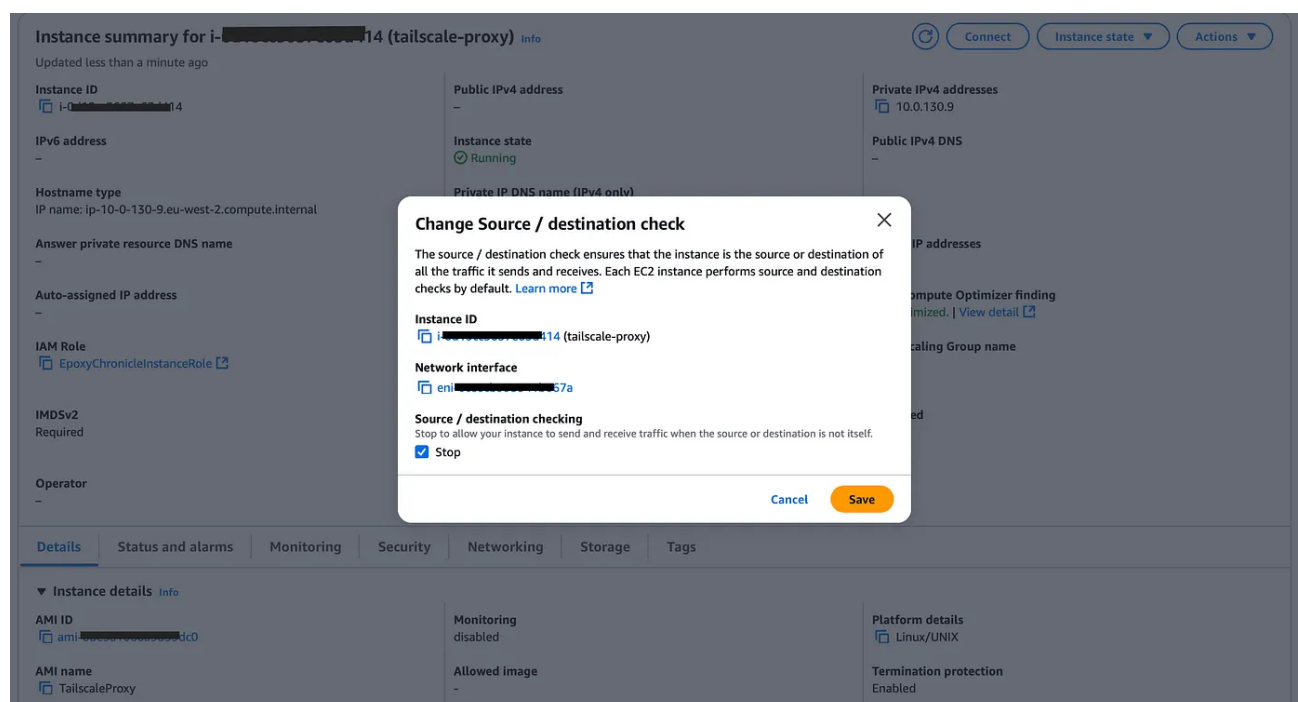
```
sudo apt update  
sudo apt install iptables-persistent
```

Then, save the current iptables rules we've applied, and enable the service so that they will be applied on every reboot.

```
sudo netfilter-persistent save  
sudo systemctl enable netfilter-persistent
```

As a final step, we need to disable source/destination checks on the EC2 instance. By default, AWS enables this check to ensure that an instance only processes traffic explicitly addressed to or from itself. However, since our instance is acting as a gateway, routing packets on behalf of other devices on our Tailscale network, we need to disable this restriction. Otherwise, AWS will silently drop any forwarded traffic.

To do this, go to the **Networking** tab of your EC2 instance, choose **Actions > Networking > Change source/dest check**, and set it to **Stop**.



Disable Source / Destination check on EC2

Configure the routing tables

Our EC2 instance is now configured to act as a routing node for the addresses advertised via Tailscale. The next step is to update the routing table so that traffic from within the VPC can be forwarded to the on-premises network through this instance.

In most setups, you'll have at least two routing tables, one for public subnets and one for private. Since we're focusing on resources in **private subnets**, open the `hybrid-rt-private` route table.

- **Local traffic** (within the VPC CIDR) should remain untouched and handled by the default `local` route.
- **Traffic destined for your on-premises network** should be routed to the EC2 **instance** acting as your Tailscale router. To do this, add a route pointing to the instance's ENI (Elastic Network Interface).
- **All other internet-bound traffic** should continue to route through the NAT Gateway, which then forwards it through the Internet Gateway (IGW).

In this way, any resource deployed within our private subnets, that are attached to this routing table, will be able to reach both on-prem and remote resources. In

my case, I've configured routes for a broader range of IPs, as I need to reach a broader range for my **hybrid Kubernetes cluster** as I've explored in depth earlier in [this article](#).

3515 / hybrid-rtb-private Actions

Details Info

Route table ID
 3515

VPC
 5fe0a | hybrid-vpc

Main

☐ No

Owner ID


Explicit subnet associations
 3 subnets

Edge associations
 -

Routes Subnet associations Edge associations Route propagation Tags

Routes (3) Both Edit routes

Filter routes

Destination	Target	Status	Propagated
0.0.0.0/0	nat-0f... 27	Active	No
10.0.0.0/16	local	Active	No
192.168.0.0/16	eni-0c... 567a	Active	No

Configure the private routing table

Configure the local resources

Now that our AWS infrastructure is in place, it's time to shift focus back to our local network. We'll configure the Raspberry Pi to join the Tailscale network and act as a gateway, forwarding local traffic destined for the AWS VPC.

Configure the Raspberry Pi

Just like we did with the EC2 instance, install Tailscale on your Raspberry Pi and bring it online. Then, to advertise your local network range (e.g., 192.168.3.0/24), use the following command:

```
sudo tailscale up --advertise-routes=192.168.3.0/24 --accept-routes
```

You can verify the Pi is connected and that both nodes (local and AWS) are part of the network.

```
pi@pi-lhr:~$ tailscale status
100.83.38.3      pi-lhr          albertox91@  linux  -
100.125.158.32  aws-exit-node   albertox91@  linux  active; relay "lhr", tx 71197908 rx 13835308
```

The Raspberry Pi has joined the Tailscale network

Now that the Raspberry Pi has joined the Tailscale mesh, and the AWS node is

advertising its VPC prefix, you should be able to ping the EC2 instance using its private IP:

```
pi@pi-lhr:~$ ping 10.0.130.9
PING 10.0.130.9 (10.0.130.9) 56(84) bytes of data.
64 bytes from 10.0.130.9: icmp_seq=1 ttl=127 time=12.7 ms
64 bytes from 10.0.130.9: icmp_seq=2 ttl=127 time=9.56 ms
64 bytes from 10.0.130.9: icmp_seq=3 ttl=127 time=10.4 ms
64 bytes from 10.0.130.9: icmp_seq=4 ttl=127 time=9.62 ms
^C
--- 10.0.130.9 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3005ms
rtt min/avg/max/mdev = 9.557/10.562/12.656/1.254 ms
pi@pi-lhr:~$ traceroute 10.0.130.9
traceroute to 10.0.130.9 (10.0.130.9), 30 hops max, 60 byte packets
 1  10.0.130.9 (10.0.130.9)  10.347 ms  10.197 ms  11.068 ms
```

Pinging the remote instance using its VPC private IP.

Since this Raspberry Pi will also act as a router for devices on your local network, it needs to be configured to forward packets over the Tailscale interface. The setup is nearly identical to the EC2 instance because in both cases, we're enabling IP forwarding and masquerading outbound packets from a LAN to Tailscale's interface (`tailscale0`).

```
echo "net.ipv4.ip_forward = 1" | sudo tee /etc/sysctl.d/99-ip-forward.conf
sudo sysctl --system
```

After enabling IPv4 forwarding, configure `iptables` to allow forwarding and apply NAT masquerading:

```
iptables -t nat -A POSTROUTING -o tailscale0 -j MASQUERADE
iptables -A FORWARD -i eth0 -o tailscale0 -j ACCEPT
iptables -A FORWARD -i tailscale0 -o eth0 -m state --state RELATED,ESTABLISHED
```

Configure the local routing tables

We're almost done. At this point, we've established a VPN tunnel between our Raspberry Pi and the EC2 instance, and configured both to act as routing gateways within their respective networks. Just like we configured the VPC

routing table to forward traffic for the on-premises network to the EC2 instance, we now need to inform our home router how to reach the AWS VPC CIDR block.

To do this, you'll need to add a **static route** on your router that tells it to send any traffic destined for the VPC (e.g., `10.0.0.0/16`) to the Raspberry Pi's local IP address (e.g., `192.168.3.5`). This essentially makes the Pi the default gateway for anything bound for the cloud. Depending on your router's interface, this configuration may vary. On my router, the static route looks like this:

- **Destination Network:** `10.0.0.0`
- **Subnet Mask:** `255.255.0.0` as the VPC has a /16 CIDR
- **Gateway:** `192.168.3.5`

Enabled	Network ↕	Subnet Mask ↕	Gateway ↕	
☒	10.0.0.0	255.255.0.0	192.168.3.5	 

My Static Route configuration on a Fritz!Box

Once this is saved, any device on your LAN will know how to reach AWS private resources, even though neither side has a public IP and no firewall changes were made.

Let's run some tests!

We're finally done.

Previously, we verified that our Raspberry Pi could reach the private IP of the EC2 instance. Now, with both the VPC and local routing tables configured to forward traffic through their respective Tailscale gateways, we've established **bidirectional connectivity** across our hybrid network.

To test this, I launched another Linux **instance** in another **private subnet** of our VPC and ran a traceroute to the **private IP address of my home router** (`192.168.3.1`). The trace shows the following path:

1. **10.0.130.9** This is the **private IP address** of the EC2 instance in the other private subnet which is acting as the **Tailscale routing node**.
2. **100.83.38.3** This is the **Tailscale-assigned IP** (from the CGNAT-style `100.x.x.x`

range) of the **Raspberry Pi** on my local network. This hop shows that the EC2 routing node successfully handed off the traffic to the Pi via Tailscale.

3. **192.168.3.1** This is my **home router**, which the Raspberry Pi forwarded the traffic to from its local ethernet interface.

```
[ssm-user@ip-10-0-150-249 bin]$ traceroute 192.168.3.1
traceroute to 192.168.3.1 (192.168.3.1), 30 hops max, 60 byte packets
 1 ip-10-0-130-9.eu-west-2.compute.internal (10.0.130.9)  11.972 ms  11.947 ms  11.913 ms
 2 100.83.38.3 (100.83.38.3)  11.882 ms  11.847 ms  11.826 ms
 3 * * *
 4 * * *
 5 * * *
 6 * * *
 7 * * *
 8 * ip-192-168-3-1.eu-west-2.compute.internal (192.168.3.1)  12.344 ms  12.322 ms
[ssm-user@ip-10-0-150-249 bin]$
```

Traceroute from AWS to Local

Interestingly, AWS's internal DNS resolver even fabricates a hostname for the private IPs, mimicking the EC2 naming convention. This is due to **reverse DNS (rDNS) resolution**, which AWS applies during the trace, regardless of whether the IP is within the VPC CIDR.

Next, I tested the reverse path by initiating a traceroute **from another Raspberry Pi on my local network to an instance in the AWS VPC**. This second Pi (named **EKS**) is part of my hybrid EKS cluster (as covered earlier [in this article](#)), and is also in the `192.168.3.0/24` subnet. The trace shows the following path:

1. **192.168.3.1** My home router, which forwards the traffic to the Pi at `192.168.3.5`, acting as our Tailscale routing node.
2. **100.125.158.32** The CGNAT-style Tailscale IP of the EC2 instance.
3. **10.0.150.249** The private IP of the target EC2 instance within the VPC.

```
pi@eks:/root$ ip a | grep 'inet 192.168.3'
    inet 192.168.3.13/24 metric 100 brd 192.168.3.255 scope global dynamic eth0
pi@eks:/root$ traceroute 10.0.150.249
traceroute to 10.0.150.249 (10.0.150.249), 30 hops max, 60 byte packets
 1 fritz.box (192.168.3.1)  0.571 ms  0.551 ms  0.693 ms
 2 192.168.3.5 (192.168.3.5)  1.246 ms  1.178 ms  1.315 ms
 3 100.125.158.32 (100.125.158.32)  12.649 ms  12.946 ms  13.026 ms
 4 10.0.150.249 (10.0.150.249)  15.185 ms  15.175 ms  15.459 ms
```

Traceroute from Local to AWS

This confirms bidirectional routing is working as expected: from AWS to your local network and back, through a fully encrypted Tailscale mesh, without the need to open any ports or manage certificates.

What's next

With this infrastructure up and running, the sky is the limit. Perhaps, you might consider deploying a hybrid **Kubernetes** cluster across AWS and your home lab, just like I did. If you're interested in this setup, I'd suggest you check out [this article](#) where I walk through my process for creating a hybrid cluster that spans both cloud and on-prem resources using Amazon **EKS Hybrid Nodes**.

But why stop there? I challenge you to take it a step further and configure a **hosted zone** for both your AWS-based services and your local devices. **Route 53's** functionality of **Outbound Endpoints** can help you achieve seamless DNS resolution across environments. You could even use something like a Raspberry Pi to host your on-prem DNS server, so that you refer to services using domains rather than IPs.

Feel free to reach out and share what you've built. I'd love to hear how you approach this challenge and any creative solutions you come up with!

Hybrid Cloud

AWS

Tailscale

Computer Networking

Hybrid Network



Follow

Written by Alberto Crescini

7 followers · 5 following

I'm Alberto, a Solutions Architect at AWS based in Hoxton, London. I hike and do bouldering. When I'm not, I'm probably feeding my sourdough starter 🍞.

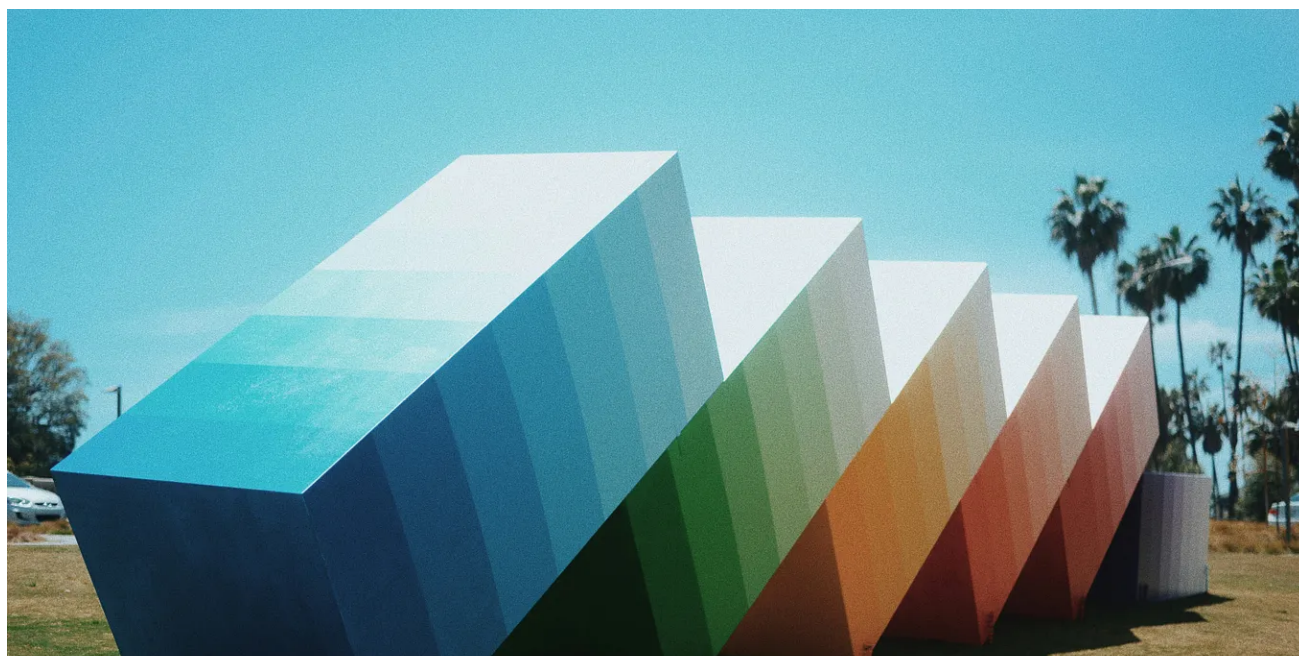
No responses yet



Write a response

What are your thoughts?

More from Alberto Crescini



Alberto Crescini

Deploy a Hybrid EKS Kubernetes Cluster: Use On-Premises nodes on Amazon EKS with Cilium CNI

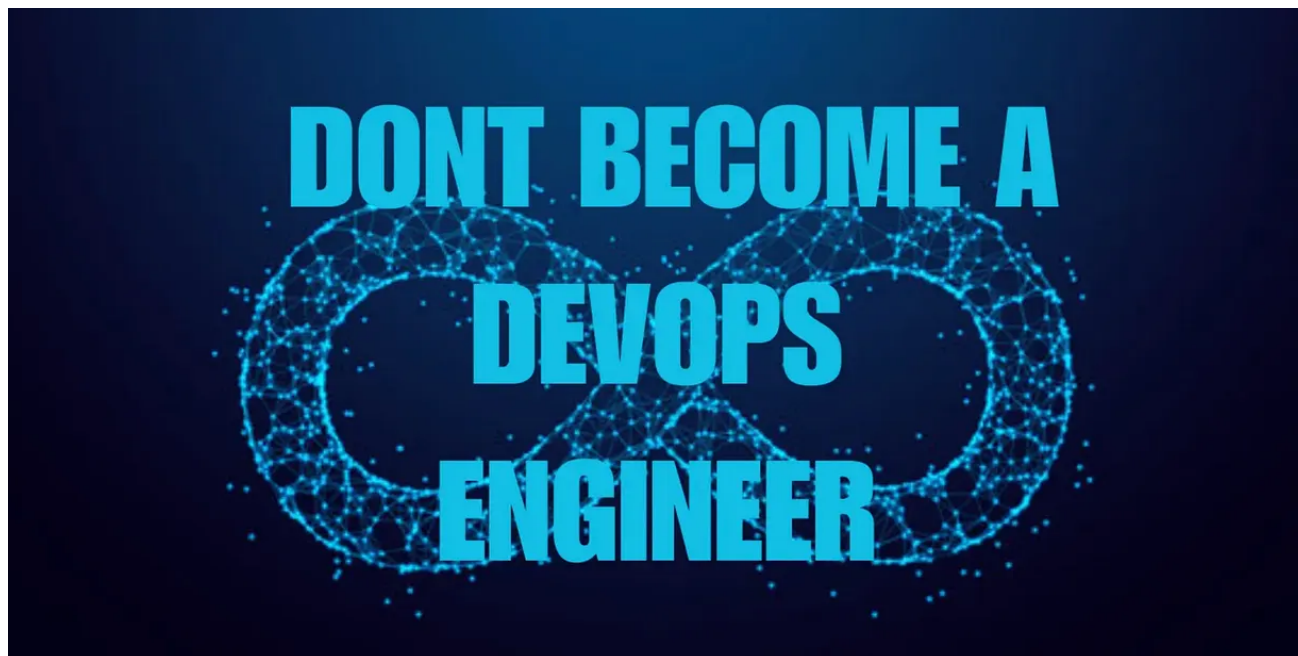
Learn to deploy a hybrid EKS Kubernetes cluster by integrating on-premises nodes with Amazon EKS using Cilium CNI for enhanced networking.

Dec 10, 2024



See all from Alberto Crescini

Recommended from Medium



 Dhanush N

Don't Become a DevOps Engineer in 2026!

Stop writing YAML. Stop babysitting pipelines. The game has fundamentally changed.

★ Mar 2 🤝 281 💬 9



 In CodeX by MayhemCode

Why Thousands Are Buying Mac Minis to Escape Issues with Big Tech AI Subscriptions Forever |...

Something strange happened in early 2026. Apple stores started running low on Mac Minis. Tech forums exploded with setup guides. Developers...

★ Feb 15  6K  105



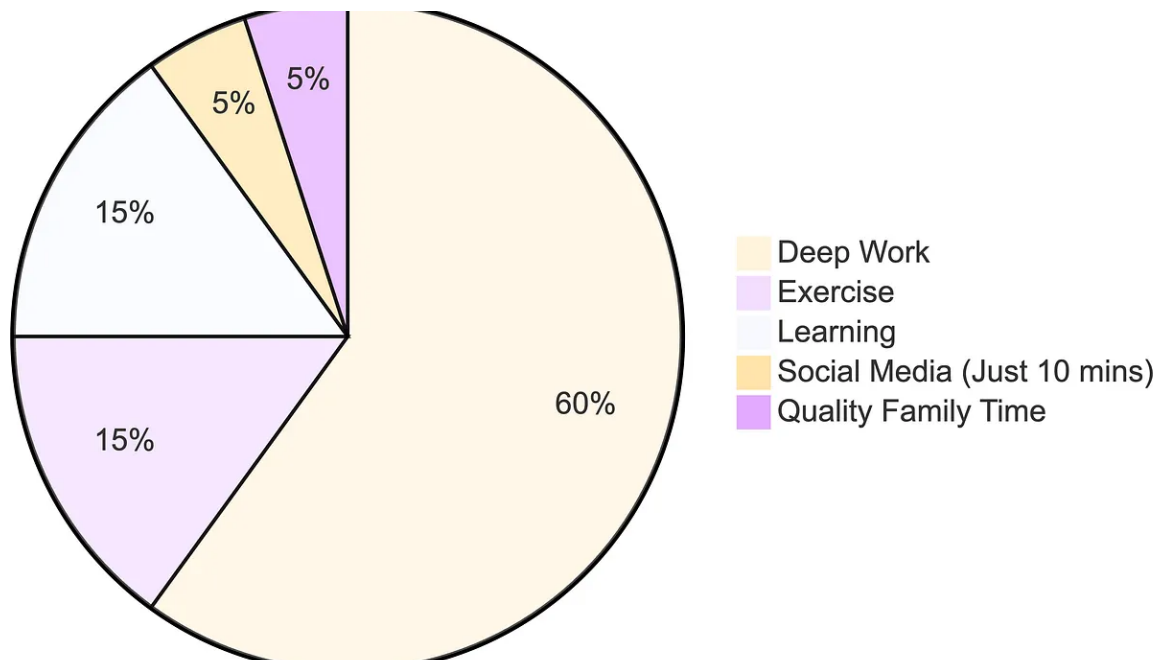
 In ILLUMINATION by Sufyan Maan, M.Eng

I Woke Up at 4:30 AM Every Day for 30 Days — Here Is What Nobody Tells You

Here is what actually happened, from someone who did it & tracked everything.

★ Mar 8  5.2K  228



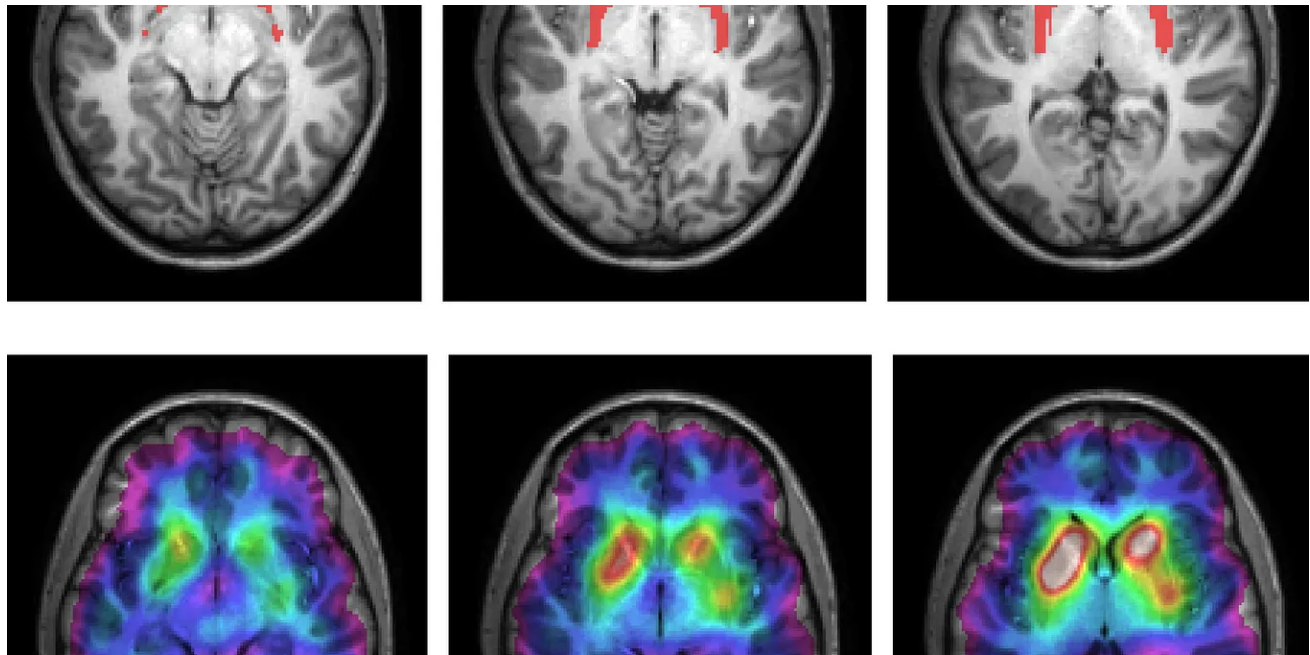


 In Level Up Coding by Kusireddy

I Stopped Using ChatGPT for 30 Days. What Happened to My Brain Was Terrifying.

91% of you will abandon 2026 resolutions by January 10th. Here's how to be in the 9% who actually win.

🌟 Dec 28, 2025 🖱 12.2K 💬 454





In Write A Catalyst by Dr. Patricia Schmidt

As a Neuroscientist, I Quit These 5 Morning Habits That Destroy Your Brain

Most people do #4 within 10 minutes of waking (and it sabotages your entire day)

ABSTRACT

👏 43K

🔥 858
Misalignment often reduces LLM diversity, leading to a phenomenon known as *mode collapse*. Unlike prior work that attributes this effect to algorithmic limitations, we identify a fundamental, pervasive data-level driver: *typicality bias* in preference data, whereby annotators systematically favor familiar text as a result of well-established findings in cognitive psychology. We formalize this bias theoretically, verify it on preference datasets empirically, and show that it plays a central role in mode collapse. Motivated by this analysis, we introduce *Verbalized Sampling (VS)*, a simple, training-free prompting strategy to circumvent mode collapse. VS prompts the model to verbalize a probability distribution over a set of responses (e.g., “Generate 5 jokes about coffee and their corresponding probabilities”). Comprehensive experiments show that VS significantly improves performance across creative writing (poems, stories, jokes), dialogue simulation, open-ended QA, and synthetic data generation, without sacrificing factual accuracy and safety. For instance, in creative writing, VS increases diversity by 1.6-2.1× over direct prompting. We further observe an emergent trend that more capable models benefit more from VS. In sum, our work provides a new data-centric perspective on mode collapse and a practical inference-time remedy that helps unlock pre-trained generative diversity.



Problem: Typicality Bias Causes Mode Collapse

Tell me a joke about coffee

Solution: Verbalized Sampling (VS) Mitigates Mode Collapse

Different prompts collapse to different modes:



In Generative AI by Adham Khaled

Stanford Just Killed Prompt Engineering With 8 Words (And I Can't Believe It Worked)

ChatGPT keeps giving you the same boring response? This new technique unlocks 2× more creativity from ANY AI model — no training required...



Oct 20, 2025



25K



690



See more recommendations